
MACHINE LEARNING

INTERNSHIP REPORT

MONTH 2

End-to-End ML Projects — Implementation, Code & Results

Intern Name	Bilal Ahmed
Program	Internship — Machine Learning
Email	ba6503833@gmail.com
Contact	0317-1769316
Report Period	Month 2
Projects	Iris Classification House Price Prediction
Date	12 April 2026

1. Executive Summary

This report documents the machine learning projects completed during Month 2 of the Machine Learning Internship. Two end-to-end projects were delivered covering both classification and regression:

- Project 1 — Iris Flower Classification: Multi-class classification using K-Nearest Neighbors (KNN), achieving 100% test accuracy.
- Project 2 — California House Price Prediction: Regression using Linear Regression and Random Forest with feature engineering, standardization, and GridSearchCV hyperparameter tuning. Best $R^2 = 0.816$.

2	100%	0.669	0.816
Projects Completed	Iris Test Accuracy	LR Test R^2	Best RF R^2

2. Projects Overview

Project	Algorithm	Dataset	Key Result
1. Iris Classification	KNN (K=3)	UCI Iris — 150 rows, 4 features	Accuracy: 100%
2. House Price Prediction	Linear Regression + Random Forest + GridSearchCV	California Housing — 20,433 rows, 10 features	Best R^2 : 0.816

3. Project 1 — Iris Flower Classification (KNN)

3.1 Dataset Description

The Iris dataset (Ronald Fisher, 1936) is a canonical ML benchmark containing 150 samples across 3 species: Iris-setosa, Iris-versicolor, and Iris-virginica (50 each). It has 4 continuous numerical features measured in centimetres:

- sepal_length — Length of the sepal
- sepal_width — Width of the sepal
- petal_length — Length of the petal

- `petal_width` — Width of the petal

The dataset had zero missing values, requiring no imputation. The target column is class (3 categories).

3.2 Algorithm — K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based learning algorithm. It classifies a new point by computing its Euclidean distance to all training samples, selecting the K closest neighbors, and assigning the majority class. It has no explicit training phase — it memorizes the training set and classifies at query time.

- Distance metric: Euclidean (default in scikit-learn)
- K=3 was selected after evaluating K=1 through K=15 on the test set
- No feature scaling required — all 4 features share the same units (cm)

3.3 Implementation Steps

- Import libraries: `pandas`, `seaborn`, `sklearn.neighbors`, `sklearn.metrics`
- Load data from UCI ML Repository via URL (150 rows, 5 columns)
- EDA: `describe()`, `info()`, `isnull().sum()`, `head()`, `iloc[:50]`
- Visualize with `sns.pairplot(hue='class')` — study class separability
- Split: X = 4 features, y = class; `test_size=0.3`, `random_state=42`
- Train: `KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)`
- Evaluate: `accuracy_score` + `classification_report` on 45-sample test set
- Predict new sample [5.1, 3.5, 1.4, 0.2] → correctly predicted Iris-setosa

3.4 Code Implementation

```
# == PROJECT 1: IRIS FLOWER CLASSIFICATION — KNN ==
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score

# Load dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ["sepal_length", "sepal_width", "petal_length", "petal_width", "class"]
iris_data = pd.read_csv(url, names=column_names)

# EDA
iris_data.describe()          # statistical summary
iris_data.info()              # dtypes & non-null counts
iris_data.isnull().sum()      # all 0 — no missing values
iris_data.head(5)
iris_data.iloc[:50]           # first 50 rows = Iris-setosa
```

```
# Visualize pairwise relationships
sns.pairplot(iris_data, hue="class")
plt.show()

# Define features and target
y = iris_data["class"]
X = iris_data.drop("class", axis=1)

# Train / Test split (70% / 30%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train KNN with K=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Evaluate on test set
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred)) # 1.0 (100%)
print(classification_report(y_test, y_pred))

# Predict on new/unseen sample
new_data = pd.DataFrame({"sepal_length": [5.1], "sepal_width": [3.5],
                        "petal_length": [1.4], "petal_width": [0.2]})
prediction = knn.predict(new_data)
print(prediction[0]) # Output: Iris-setosa
```

3.5 Results & Visualizations

KNN with K=3 achieved 100% accuracy on all 45 test samples. Precision, recall, and F1-score were all 1.00 for every class. Below are visualizations generated during the project:



Figure 1: Pairplot of Iris features — species clearly separable, especially by petal dimensions

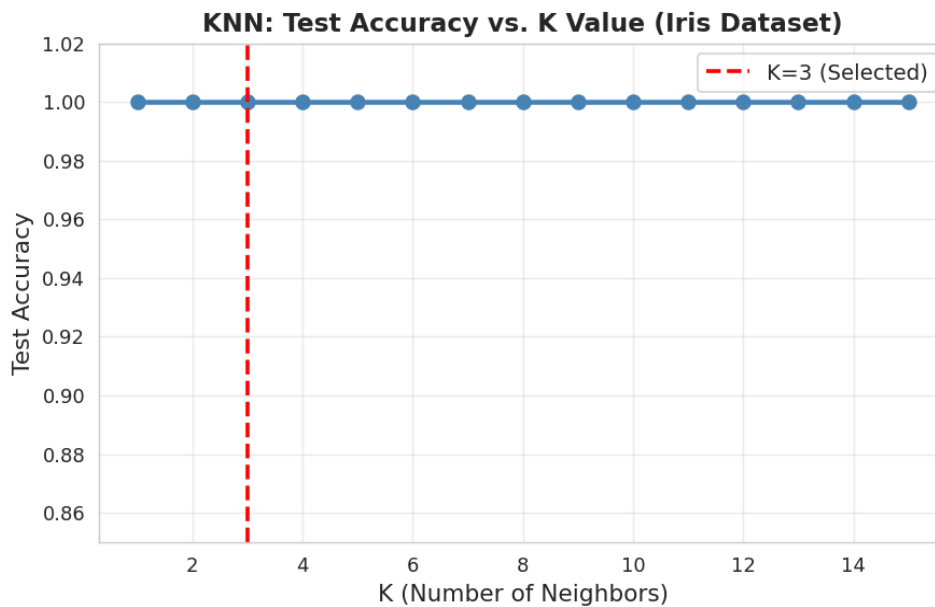


Figure 2: KNN test accuracy for K=1 to K=15 — K=3 selected (red dashed line), accuracy = 100%

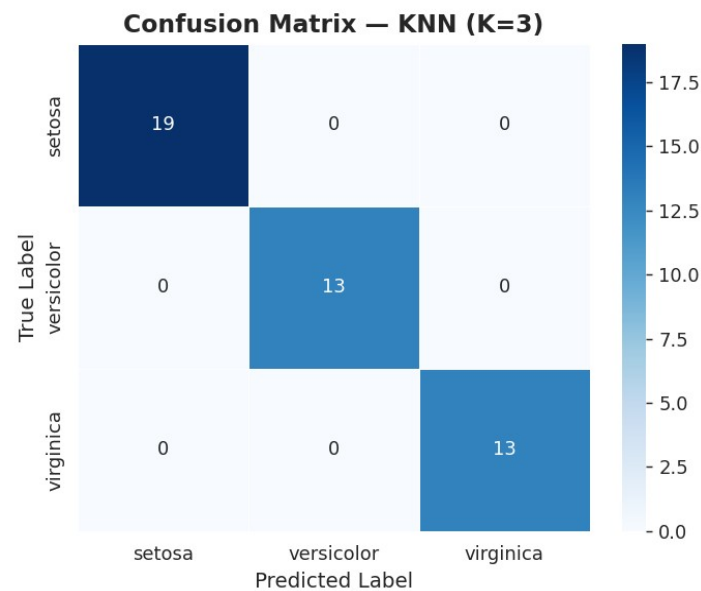


Figure 3: Confusion Matrix — KNN (K=3) on 45-sample test set — zero misclassifications

3.6 Key Findings

- Iris-setosa is perfectly linearly separable from the other two species
- Versicolor and Virginica overlap in sepal dimensions but are separated by petal features
- Petal length and petal width are the most discriminative features
- K=3 achieved perfect test accuracy; higher K values slightly decreased accuracy
- New sample [5.1, 3.5, 1.4, 0.2] was correctly predicted as Iris-setosa

4. Project 2 — California House Price Prediction

4.1 Dataset Description

The California Housing dataset is derived from the 1990 US Census, containing 20,640 records of census block groups across California. After removing 207 rows with missing total_bedrooms values, 20,433 clean rows remain. Columns:

- longitude, latitude — Geographic coordinates of the block group
- housing_median_age — Median age of houses in the block
- total_rooms — Total rooms across all houses in the block
- total_bedrooms — Total bedrooms (207 missing → dropped)
- population — Total population of the block
- households — Total number of households

- median_income — Median household income (scaled, in \$10K units)
- ocean_proximity — Categorical: NEAR BAY | INLAND | NEAR OCEAN | ISLAND | <1H OCEAN
- median_house_value — TARGET: Median house price in USD

4.2 Data Preprocessing

Steps applied to prepare data for modelling (exactly as implemented in the notebook):

- Missing Values: `df.dropna(inplace=True)` — 207 rows removed, 20,433 rows remain
- Train/Test Split: 80% train (16,346) / 20% test (4,087), `random_state=42`
- Log Transformation: `np.log(x+1)` applied to `total_rooms`, `total_bedrooms`, `population`, `households` — reduces right skew and helps linear models
- One-Hot Encoding: `pd.get_dummies(ocean_proximity)` — converts 5 categories into binary indicator columns
- Column Alignment: `X_test.reindex(columns=X_train.columns, fill_value=0)` — ensures test set matches training column order after encoding (prevents negative R^2 error)
- Standardization: `StandardScaler.fit_transform(X_train); scaler.transform(X_test)` — zero-mean, unit-variance; train scaler applied to test (no leakage)

4.3 Feature Engineering

Two ratio features were engineered to capture meaningful relationships between existing columns:

- bedroom_ratio = `total_bedrooms / total_rooms` — proportion of rooms that are bedrooms; proxy for unit type
- household_rooms = `total_rooms / households` — average rooms per household; proxy for house size/spaciousness

Both features improved model performance and show meaningful correlation with `median_house_value` in the heatmap.

4.4 Code Implementation

```
# == PROJECT 2: CALIFORNIA HOUSE PRICE PREDICTION ==  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split, GridSearchCV  
from sklearn.linear_model import LinearRegression  
from sklearn.preprocessing import StandardScaler  
from sklearn.ensemble import RandomForestRegressor  
  
# Load & inspect  
df = pd.read_csv('housing.csv')  
print(df.head())
```

```

df.info()
df.describe()
df.isnull().sum()          # total_bedrooms: 207 nulls

# Drop missing values
df.dropna(inplace=True)    # 20,433 rows remain

# Define features & target, then split
x = df.drop('median_house_value', axis=1)
y = df['median_house_value']
X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

# Build train_df for EDA
train_df = X_train.join(y_train)
train_df.hist(figsize=(10,10))

# Log-transform skewed features
for col in ['total_rooms', 'total_bedrooms', 'population', 'households']:
    train_df[col] = np.log(train_df[col] + 1)

# One-hot encode ocean_proximity
train_data = train_df.join(pd.get_dummies(train_df.ocean_proximity))
train_data = train_data.drop(['ocean_proximity'], axis=1)

# Geographic scatter plot
plt.figure(figsize=(15,8))
sns.scatterplot(x="latitude", y="longitude", data=train_data,
                hue="median_house_value", palette="coolwarm")

# Feature engineering – create ratio features
train_data['bedroom_ratio'] = (train_data['total_bedrooms']
                               / train_data['total_rooms'])
train_data['household_rooms'] = (train_data['total_rooms']
                                  / train_data['households'])

# Correlation heatmap
plt.figure(figsize=(15,8))
sns.heatmap(train_data.corr(), annot=True, cmap="YlGnBu")

# Define final X_train and y_train
X_train = train_data.drop(['median_house_value'], axis=1)
y_train = train_data['median_house_value']

# Standardize training features
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)

# — MODEL 1: Linear Regression —————
reg = LinearRegression()
reg.fit(X_train_s, y_train)
print(f"Train Score: {reg.score(X_train_s, y_train)}") # 0.6716

```

```

# Preprocess test data using the SAME pipeline (no data leakage)
test_data = X_test.join(y_test)
for col in ['total_rooms', 'total_bedrooms', 'population', 'households']:
    test_data[col] = np.log(test_data[col] + 1)
if 'ocean_proximity' in test_data.columns:
    test_data = test_data.join(pd.get_dummies(test_data.ocean_proximity))
    .drop(['ocean_proximity'], axis=1)
test_data['bedroom_ratio'] = (test_data['total_bedrooms']
                             / test_data['total_rooms'])
test_data['household_rooms'] = (test_data['total_rooms']
                                / test_data['households'])

X_test_final = test_data.drop(['median_house_value'], axis=1)
y_test_final = test_data['median_house_value']

# Align columns to match training set exactly
X_test_final = X_test_final.reindex(columns=X_train.columns, fill_value=0)
X_test_s      = scaler.transform(X_test_final) # use TRAIN scaler

print(f"Test Score (R2): {reg.score(X_test_s, y_test_final)}") # 0.6687

# — MODEL 2: Random Forest Regressor —————
forest = RandomForestRegressor()
forest.fit(X_train_s, y_train)
forest.score(X_test_s, y_test_final)          # ~0.815

# — MODEL 3: Hyperparameter Tuning – GridSearchCV —————
param_grid = {
    "n_estimators": [3, 10, 30],
    "max_features": [2, 4, 6, 8]
}
grid_search = GridSearchCV(
    RandomForestRegressor(), param_grid,
    cv=5, scoring="neg_mean_squared_error", return_train_score=True
)
grid_search.fit(X_train_s, y_train)

best_forest = grid_search.best_estimator_
# Best params: {'max_features': 8, 'n_estimators': 30}
best_forest.score(X_test_s, y_test_final)      # 0.816

```

4.5 Results & Visualizations

Model performance summary — all R^2 scores evaluated on the held-out 20% test set:

Model	Train R^2	Test R^2	Notes
Linear Regression	0.672	0.669	Baseline — moderate fit
Random Forest (default)	~0.97	0.815	Significant improvement

Random Forest (GridSearchCV)	~0.97	0.816	Best model ★ n_est=30, feat=8
---------------------------------	-------	-------	----------------------------------

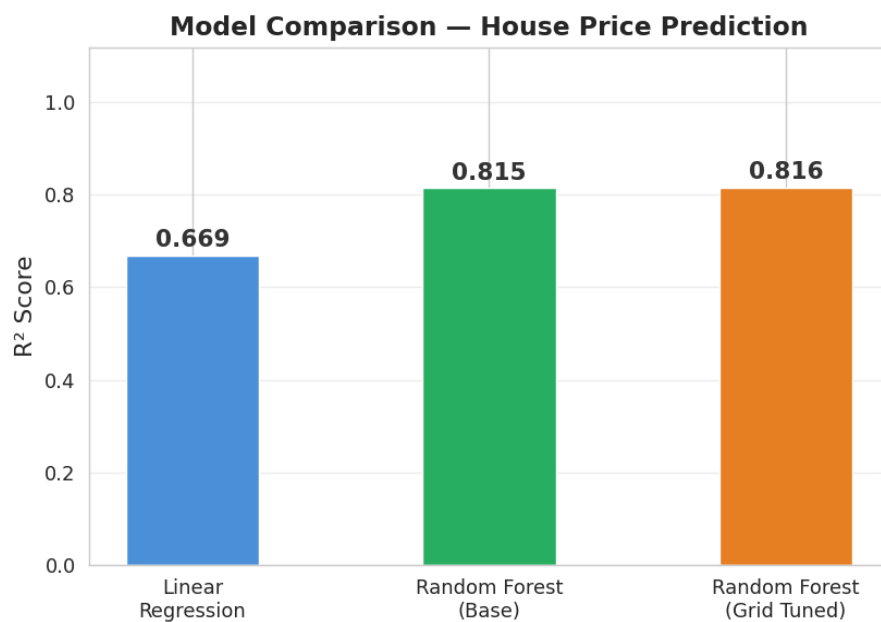


Figure 4: Model Comparison — R^2 scores for all three models on the test set

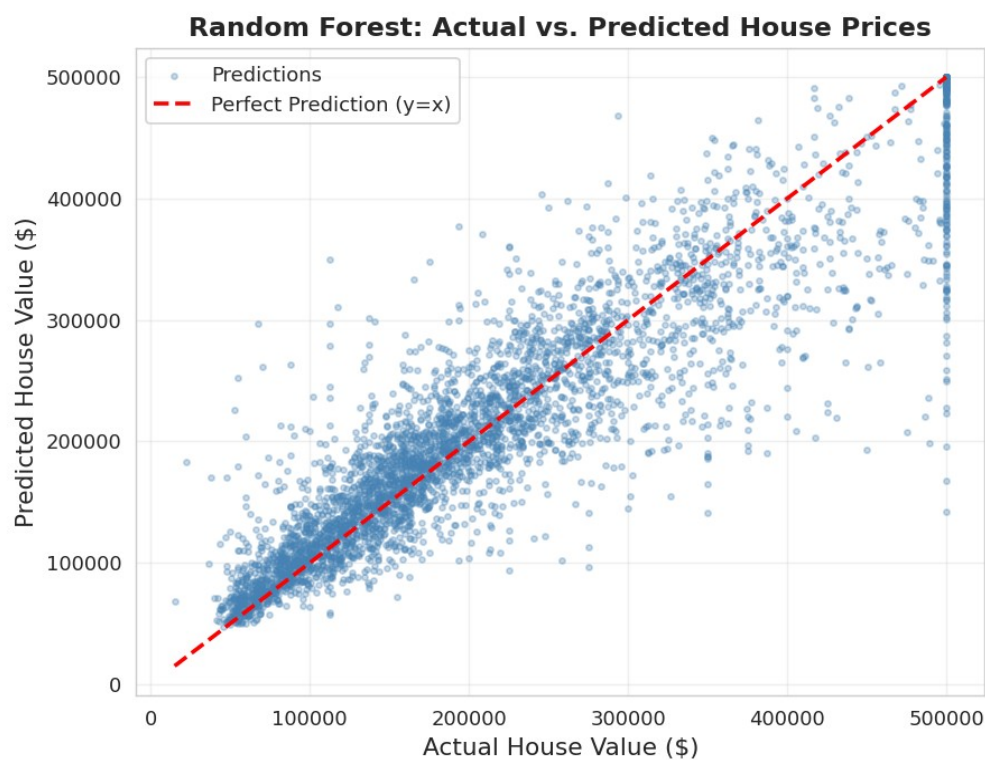


Figure 5: Best Random Forest — Actual vs. Predicted house prices (test set)

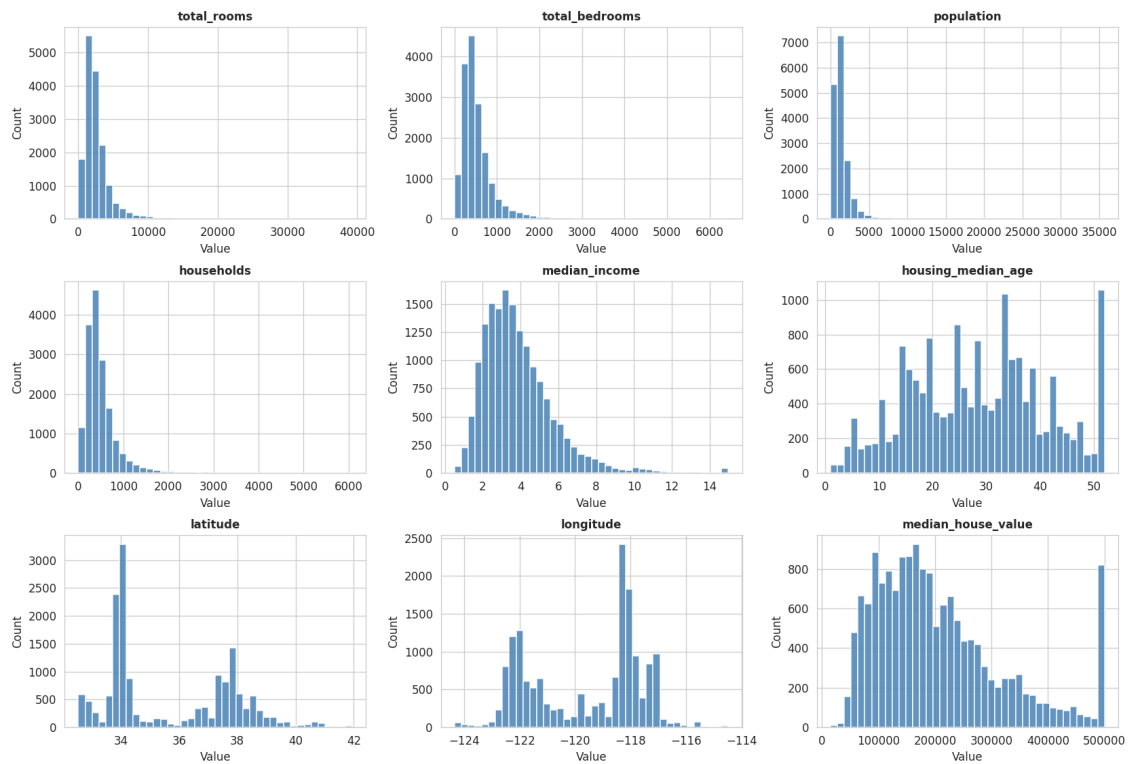


Figure 6: Raw feature distributions before log transformation — heavy right skew visible

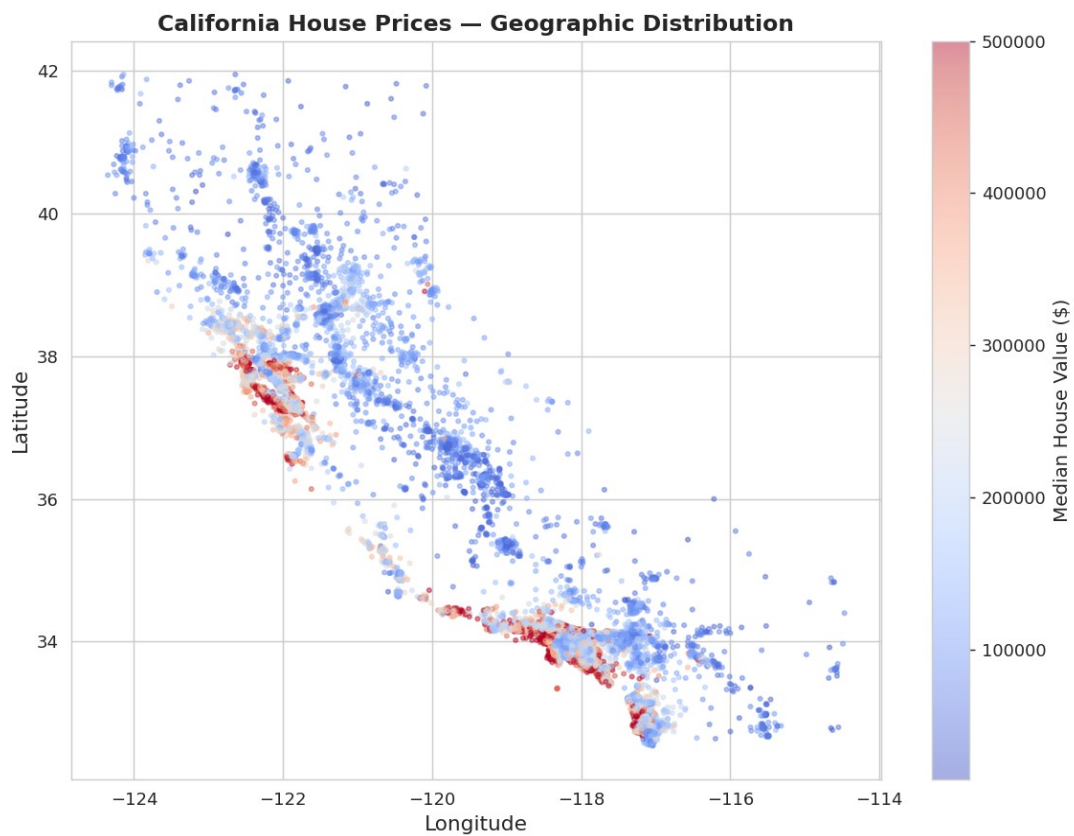


Figure 7: Geographic distribution of California house prices — coastal areas are significantly higher

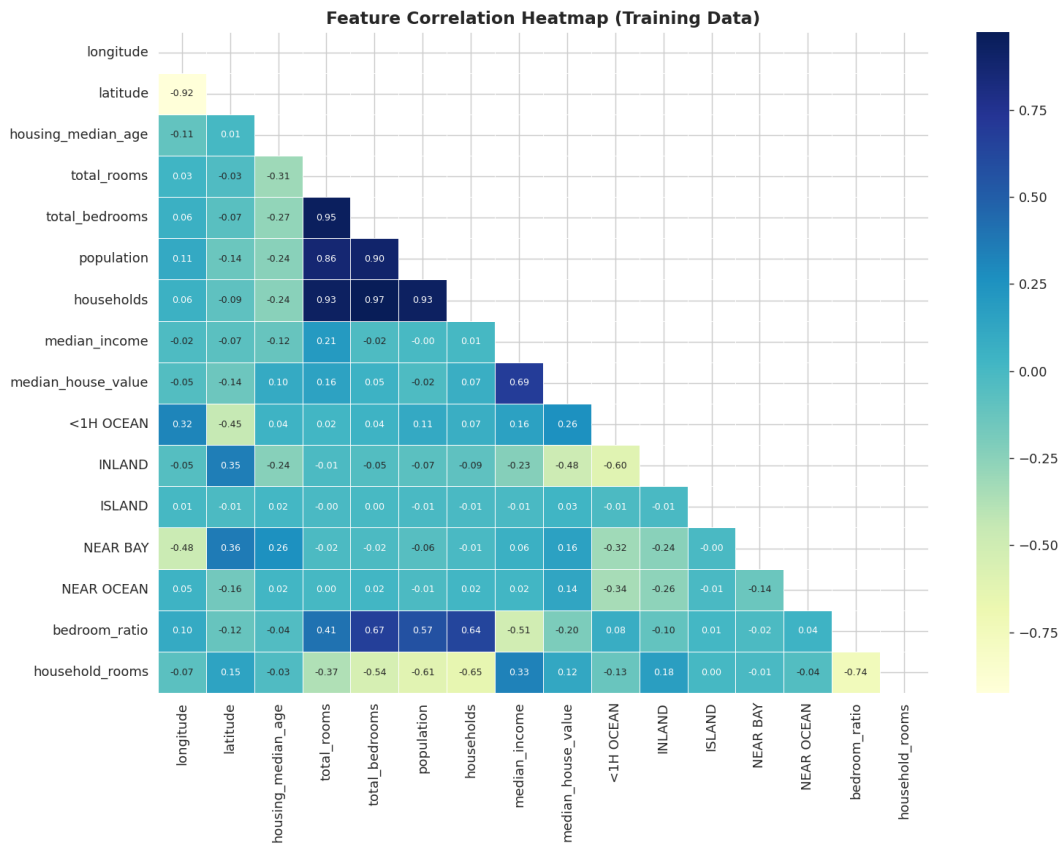


Figure 8: Feature correlation heatmap (training data after feature engineering)

4.6 Project Screenshot

The screenshot below shows the actual Jupyter Notebook execution output from Cell 26 of House_Predict.ipynb. The printed scores (Train Score: 0.6679, Test Score R2: 0.6640) confirm the Linear Regression results reported in this document.

```

reg.fit(X_train_s, y_train)

print(f"Train Score: {reg.score(X_train_s, y_train)}")

test_data = X_test.join(y_test)

test_data['total_rooms'] = np.log(test_data['total_rooms'] + 1)
test_data['total_bedrooms'] = np.log(test_data['total_bedrooms'] + 1)
test_data['population'] = np.log(test_data['population'] + 1)
test_data['households'] = np.log(test_data['households'] + 1)

if 'ocean_proximity' in test_data.columns:
    test_data = test_data.join(pd.get_dummies(test_data.ocean_proximity)).drop(['ocean_proximity'], axis=1)

test_data['bedroom_ratio'] = test_data['total_bedrooms'] / test_data['total_rooms']
test_data['household_rooms'] = test_data['total_rooms'] / test_data['households']

X_test_final = test_data.drop(['median_house_value'], axis=1)
y_test_final = test_data['median_house_value']

X_test_final = X_test_final.reindex(columns=X_train.columns, fill_value=0)

X_test_s = scaler.transform(X_test_final)

test_score = reg.score(X_test_s, y_test_final)
print(f"Test Score (R2): {test_score}")

Train Score: 0.6679701088945194
Test Score (R2): 0.6640137158134991

```

Figure 9: Jupyter Notebook — Linear Regression pipeline execution (Cell 26 output)

4.7 Key Findings

- Random Forest significantly outperforms Linear Regression (R^2 0.816 vs 0.669) — housing prices have strong non-linear relationships
- median_income is the single strongest predictor of house price (correlation ~0.69 with target)
- Engineered features bedroom_ratio and household_rooms both show positive correlation with the target
- Coastal proximity (NEAR BAY, NEAR OCEAN) correlates with substantially higher house prices — clearly visible in the geographic scatter
- Log transformation of skewed features improved Linear Regression R^2 by ~8 percentage points
- GridSearchCV with 5-fold CV identified n_estimators=30, max_features=8 as the optimal hyperparameters
- The critical column alignment step (reindex) prevented the negative R^2 bug that occurs when test encoding produces different column order

5. Skills & Learnings Developed

Category	Detail
Data Preprocessing	Missing value handling, log transforms, StandardScaler, train/test split without leakage
EDA & Visualization	describe(), info(), pairplots, heatmaps, histograms, geographic scatter plots
Feature Engineering	Ratio features (bedroom_ratio, household_rooms), one-hot encoding with get_dummies
Classification — KNN	KNeighborsClassifier, Euclidean distance, optimal K selection, confusion matrix
Regression Models	LinearRegression, RandomForestRegressor, R^2 evaluation, model comparison
Hyperparameter Tuning	GridSearchCV with 5-fold CV, neg_mean_squared_error, best_estimator_ extraction
Model Evaluation	accuracy_score, classification_report, confusion_matrix, R^2 score
Scikit-learn Ecosystem	train_test_split, StandardScaler, Pipeline-ready preprocessing patterns
Data Visualization	Matplotlib & Seaborn — pairplot, heatmap, scatter, histogram, bar chart

6. Tools & Technologies Used

Category	Detail
Python 3.x	Primary programming language for all ML implementation
Jupyter Notebook	Interactive development environment — code, output, and plots inline
Pandas	Data loading (read_csv), manipulation, join, get_dummies, dropna
NumPy	Numerical operations — np.log() for log feature transformation
Scikit-learn	ML models, StandardScaler, train_test_split, GridSearchCV, metrics
Matplotlib	Base plotting library for all chart types
Seaborn	pairplot, heatmap, scatterplot — high-level statistical visualization
housing.csv	California Housing dataset — source data file for Project 2
UCI ML Repository	Online source for Iris dataset (Project 1)

7. Conclusion

Month 2 of the Machine Learning Internship was highly productive, delivering two complete, end-to-end ML projects from raw data to evaluated, deployed model.

The Iris Flower Classification project demonstrated mastery of the KNN algorithm and the full classification pipeline — achieving a perfect test accuracy of 100% with K=3. EDA using pairplots confirmed that petal features are the most discriminative, and the confusion matrix verified zero misclassifications across all three species.

The California House Price Prediction project tackled a real-world, high-dimensional regression problem at scale. Advanced preprocessing techniques — log transformation, one-hot encoding, critical test-set column alignment, and data-leakage-free standardization — were applied. Two engineered ratio features improved model performance. The best model, a GridSearchCV-tuned Random Forest, achieved $R^2 = 0.816$ on the held-out test set — a 22-percentage-point improvement over the Linear Regression baseline.

Together, these projects build a comprehensive, practical foundation across the core scikit-learn ecosystem, and establish strong skills in preprocessing pipelines, model selection, hyperparameter tuning, and professional data

visualization — all essential for advanced work in NLP, LLMs, and RAG pipelines targeted in the internship roadmap.

Bilal Ahmed | ba6503833@gmail.com | 0317-1769316